

Code:

```
#include <stdio.h> #include <stdlib.h> #include <string.h>

const int MAXL = 200005;

struct state { int len, link; int next[64]; };

struct state st[200005]; int sz, last;

void sam_init() { st[0].len = 0; st[0].link = -1; memset(st[0].next, -1, sizeof(st[0].next)); sz = 1; last = 0; }

void sam_extend(int c) { int cur = sz++; st[cur].len = st[last].len + 1; memset(st[cur].next, -1, sizeof(st[cur].next)); int p = last; while (p != -1 && st[p].next[c] == -1) { st[p].next[c] = cur; p = st[p].link; } if (p == -1) { st[cur].link = 0; } else { int q = st[p].next[c]; if (st[p].len + 1 == st[q].len) { st[cur].link = q; } else { int clone = sz++; st[clone].len = st[p].len + 1; memcpy(st[clone].next, st[q].next, sizeof(st[q].next)); st[clone].link = st[q].link; while (p != -1 && st[p].next[c] == q) { st[p].next[c] = clone; p = st[p].link; } st[q].link = st[cur].link = clone; } } last = cur; }

int head[MAXL], to[2 * MAXL], nxt[2 * MAXL], edge_cnt; int deg[MAXL]; int parent_node[MAXL];

void add_edge(int u, int v) { to[++edge_cnt] = v; nxt[edge_cnt] = head[u]; head[u] = edge_cnt; }

void dfs(int u, int p) { parent_node[u] = p; for (int e = head[u]; e; e = nxt[e]) { int v = to[e]; if (v != p) { dfs(v, u); } } }

int main() { int n; if (scanf("%d", &n) != 1) return 0;

sam_init();
for (int i = 0; i < n - 1; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    add_edge(u, v);
    add_edge(v, u);
    deg[u]++;
    deg[v]++;
}

dfs(1, 0);
```

```
for (int i = 1; i <= n; i++) {
    last = 0;
    int curr = i;
    while (curr != 0) {
        sam_extend(deg[curr]);
        curr = parent_node[curr];
    }
}

long long total_distinct_trips = 0;
for (int i = 1; i < sz; i++) {
    total_distinct_trips += st[i].len - st[st[i].link].len;
}

printf("%lld\n", total_distinct_trips);
return 0;

}
```

Input:

```
3
2 1
3 1
```

Output:

```
3
```